

Projet Final

Construction d'un pipeline ETL avec Apache Airflow, Elasticsearch et Kibana

Dépôt du projet avant le : 06 octobre 2026

1. Contexte

Une entreprise souhaite mettre en place une plateforme permettant de collecter, transformer, analyser et visualiser des données provenant de sources variées.

Votre mission consiste à construire un **pipeline ETL automatisé avec Apache Airflow**. Les données seront ensuite stockées dans **Elasticsearch**, analysées et visualisées à l'aide de **Kibana**.

Le projet doit représenter un cas concret de Data Engineering. Chacun choisira un domaine parmi les exemples suivants :

- e-commerce et commandes ;
- produits et avis clients ;
- films, séries ou musique ;
- transport et mobilité ;
- météo et environnement ;
- données sportives ;
- actualités provenant d'une API publique.
-

2. Objectif du projet

L'objectif est de réaliser toute la chaîne suivante :

Collecter → Transformer → Contrôler → Charger → Analyser → Visualiser

À la fin du projet, vous devez être capable de :

1. récupérer automatiquement des données ;
2. nettoyer et transformer les données ;
3. contrôler leur qualité ;
4. charger les données dans Elasticsearch ;
5. orchestrer l'ensemble avec Apache Airflow ;
6. réaliser des analyses à partir des données ;
7. construire un tableau de bord interactif avec Kibana.

Le projet doit être **reproductible** : une autre personne doit pouvoir cloner le dépôt GitHub et lancer le projet en suivant le README.

3. Architecture attendue

L'architecture minimale du projet est :

API / CSV / JSON → Airflow → Extract → Transform → Validate → Elasticsearch → Kibana

4. Fonctionnalités Airflow à explorer

Le DAG doit utiliser au minimum :

- un **DAG** avec une planification ;
- plusieurs tâches avec des dépendances ;
- **PythonOperator** ou opérateur équivalent ;
- une connexion Airflow vers Elasticsearch ;
- les **retries** en cas d'échec ;
- les **logs** pour suivre les exécutions ;
- **XCom** lorsque son utilisation est pertinente ;
- une tâche de contrôle de la qualité des données.

Vous pouvez explorer d'autres fonctionnalités d'Airflow : Variables, Connections, Task Groups, templates, paramètres ou notifications.

5. Travail demandé

Le pipeline doit réaliser les étapes suivantes :

1. **Extraction** : récupérer les données depuis une API, un fichier CSV/JSON ou plusieurs sources.
2. **Transformation** : réaliser au moins trois transformations significatives : suppression des doublons, gestion des valeurs manquantes, conversion de types, normalisation des dates, filtrage, agrégation, création de nouvelles colonnes ou jointure.
3. **Validation** : vérifier la qualité des données, par exemple le nombre de lignes, les champs obligatoires, les types ou l'unicité d'un identifiant.
4. **Chargement** : créer un index Elasticsearch, définir si nécessaire le mapping des champs et charger les documents.
5. **Analyse** : produire plusieurs indicateurs pertinents à partir des données. Les analyses doivent répondre à des questions concrètes liées au domaine choisi.
6. **Visualisation** : créer dans Kibana au minimum **4 visualisations différentes** et un **dashboard** regroupant les principaux indicateurs.

6. Organisation du dépôt GitHub

Le dépôt devra être organisé clairement :

```
projet-data-engineering/  
  dags/  
    etl_pipeline.py  
  scripts/  
    extract.py  
    transform.py  
    validate.py  
    load.py  
  data/  
    elasticsearch/  
      mapping.json  
      queries.json  
    kibana/  
      dashboard.ndjson  
  docker-compose.yml  
  requirements.txt  
  README.md
```

Le fichier **README.md** doit présenter le contexte, l'architecture, les sources, les transformations, le mapping Elasticsearch, les analyses, le dashboard Kibana et les instructions permettant de lancer le projet.

Des captures d'écran du DAG Airflow et du dashboard Kibana devront également être présentées dans le README.

7. Livrables

Chaque étudiant doit déposer son travail sur GitHub :

- le code source complet ;
- le DAG Airflow ;
- les données ou les instructions pour les récupérer ;
- la configuration Docker ;
- la configuration et les requêtes Elasticsearch ;
- les contrôles de qualité ;
- le dashboard Kibana ;
- un README complet ;
- une courte démonstration du projet.

8. Conseil

Ne cherchez pas à construire un système trop complexe. Un projet **simple, fonctionnel, automatisé et bien documenté** est préférable à un projet très ambitieux mais difficile à reproduire.

À la fin du projet, vous devez pouvoir expliquer simplement le parcours d'une donnée :

Comment la donnée est-elle collectée, transformée, contrôlée, stockée, analysée et finalement visualisée ?